

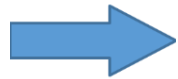
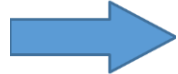
LISTS AND MUTABILITY

Aliasing, copies, and methods that change the list
the list

== VS is

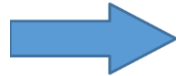
b is a lookalike: same contents, separate list

== asks “same value?” **is** asks “same object?”



```
1 a = [1, 2]
2 b = [1, 2]
3 c = a
4 print(a == b, a is b, a is c)
5
6 c.append(3)
7 print(a)
```

c is an alias for **a**, so **a** sees the change



Output

```
True False True
[1, 2, 3]
```

id(x) gives an object's identity. **a is b** means **id(a) == id(b)**.
== id(b).

TRY IT: == VS is

A · Predict each line

```
1 x = [4, 5]
2 y = x
3 z = [4, 5]
4 print(x == z, x is z)
5 print(x is y)
6 y.append(6)
7 print(x == z)
```

What prints? Which names are the same list?

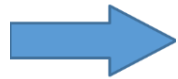
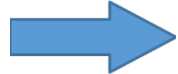
B · Predict each line

```
1 a = [1, 2, 3]
2 b = a[:]
3 print(a == b, a is b)
4 b[0] = 99
5 print(a)
```

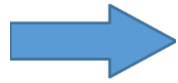
Does changing **b** change **a**? Why or why not?

LISTS WITHIN LISTS

grid stores a reference to **row**, not a copy
copy
Reaches into the inner list and changes it
it

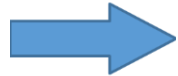


row and **grid[0]** are the same list



```
1 row = [1, 2]
2 grid = [row, [3, 4]]
3
4 grid[0][1] = 99
5 print(row)
6
7 row.append(5)
8 print(grid)
```

So **grid** sees the change too



Output

```
[1, 99]
[[1, 99, 5], [3, 4]]
```

TRY IT: LISTS WITHIN LISTS

A · Predict the output

```
1 a = [1, 2]
2 b = [a, a]
3 b[0].append(3)
4 print(b)
```

How many different inner lists does **b** really hold?

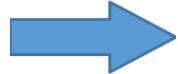
B · Predict, then fix

```
1 rows = []
2 row = []
3 for i in range(3):
4     row.append(i)
5     rows.append(row)
6 print(rows)
```

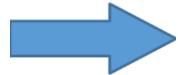
Fix it so it prints **[[0], [0, 1], [0, 1, 2]]**

SHALLOW COPIES

a.copy(), **a[:]** and **list(a)** make a new outer list, but the items inside are shared

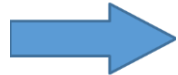


Changing the outer list: only the copy changes



```
1 grid = [[1, 2], [3, 4]]
2 shallow = grid.copy()
3
4 shallow.append([5, 6])
5 shallow[0][0] = 99
6
7 print(grid)
8 print(shallow)
```

The inner lists were shared, so **grid** changed too



Output

```
[[99, 2], [3, 4]]
[[99, 2], [3, 4], [5, 6]]
```

With only numbers or strings inside, a shallow copy is all you need.

TRY IT: SHALLOW COPIES

A · Predict the output

```
1 nums = [1, 2, 3]
2 backup = nums.copy()
3 backup[0] = 100
4 print(nums)
5 print(backup)
```

Is a shallow copy enough here? Why?

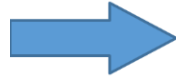
B · Predict the output

```
1 seats = [['Alice', 'Bob'], ['Chris']]
2 saved = seats[:]
3 saved[1].append('Dave')
4 saved.append(['Eve'])
5 print(seats)
6 print(saved)
```

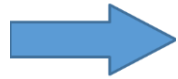
Which change shows up in both lists, and which doesn't?

import copy

copy is built into Python. Nothing to install.
install.

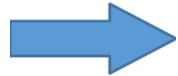


Copies the outer list and every list inside it,
inside it, as deep as they go



```
1 import copy
2
3 grid = [[1, 2], [3, 4]]
4 deep = copy.deepcopy(grid)
5
6 deep[0][0] = 99
7 print(grid)
8 print(deep)
```

grid is untouched



copy.copy(x) also exists. It's a shallow copy, the same as
same as **x.copy()** for a list.

Output

```
[[1, 2], [3, 4]]
[[99, 2], [3, 4]]
```

TRY IT: import copy

A · Fix it

```
1 grid = [[1, 2], [3, 4]]
2 backup = grid.copy()
3 backup[0][0] = 99
4 print(grid)
5 # want: [[1, 2], [3, 4]]
```

Change the code so **backup** is completely independent of **grid**.

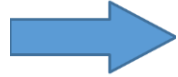
B · Predict the output

```
1 import copy
2 a = [[0], [0]]
3 b = copy.copy(a)
4 c = copy.deepcopy(a)
5 a[0].append(1)
6 print(b)
7 print(c)
```

Which copy sees the change? Why?

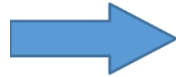
THE MULTIPLICATION TRAP

* repeats references: three slots, one inner list

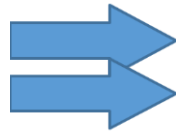


```
1 board = [[0] * 3] * 3
2 board[0][0] = 'X'
3 print(board)
4
5 board = [[0] * 3 for _ in range(3)]
6 board[0][0] = 'X'
7 print(board)
```

A comprehension builds a fresh row on each on each pass



Every row changed
Only the first row changed



Output

```
[['X', 0, 0], ['X', 0, 0], ['X', 0, 0]]
[['X', 0, 0], [0, 0, 0], [0, 0, 0]]
```

[0] * 3 on its own is fine: row[0] = 5 just points one slot at a slot at a new number.

TRY IT: THE MULTIPLICATION TRAP

A · Predict the output

```
1 row = [0] * 3
2 row[1] = 7
3 print(row)
4 grid = [row] * 2
5 grid[0][0] = 5
6 print(grid)
```

Why is the first print fine but the second surprising?

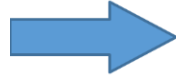
B · Predict, then fix

```
1 teams = [[]] * 3
2 teams[0].append('Alice')
3 print(teams)
```

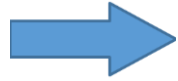
Fix it so only team 0 gets Alice.

METHODS THAT CHANGE THE LIST

sort() changes the list itself and returns **None**



sorted() leaves **nums** alone and returns a new list



Also in place: **append**, **extend**, **insert**, **remove**, **reverse**, **reverse**, **clear**, and **+=** on a list. So never write **nums = nums.sort()**.

```
1 nums = [3, 1, 2]
2 result = nums.sort()
3 print(result)
4 print(nums)
5
6 nums = [3, 1, 2]
7 ordered = sorted(nums)
8 print(nums)
9 print(ordered)
```

Output

```
None
[1, 2, 3]
[3, 1, 2]
[1, 2, 3]
```

TRY IT: IN-PLACE METHODS

A · Predict the output

```
1 nums = [5, 3, 8]
2 nums = nums.append(1)
3 print(nums)
```

Where did the list go?

B · Predict the output

```
1 a = [3, 1, 2]
2 b = a
3 b.sort()
4 b = b + [4]
5 print(a)
6 print(b)
```

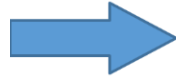
Which line changed **a**, and which didn't?

C · Sort these: in place, or new list?

```
sort()    sorted()    append()    +    reverse()   [::-1]    extend()    copy()
```

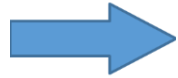
PASSING A LIST TO A FUNCTION

scores is a parameter: a new name for
for whatever list the caller passes in

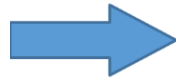


```
1 def add_bonus(scores):  
2     scores.append(10)  
3  
4 marks = [7, 8]  
5 add_bonus(marks)  
6 print(marks)
```

This call makes **scores** an alias for **marks**,
just like **c = a**



append changed the one shared list, so
so **marks** sees it



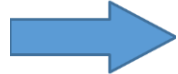
Output

```
[7, 8, 10]
```

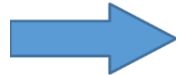
Python never copies a list when you pass it in. Inside the
Inside the call, **scores is marks** is True.

CHANGE IT OR REPLACE IT?

clear() changes the shared list, so the caller sees it



items = [] only moves the local name to a new list. **nums** is untouched



Changing the list (**append**, **clear**, **x[i] = ...**, **+=**) reaches the caller. Using **=** on the parameter name does not.

```
1 def empty_it(items):
2     items.clear()
3
4 def replace_it(items):
5     items = []
6
7 nums = [1, 2]
8 replace_it(nums)
9 print(nums)
10 empty_it(nums)
11 print(nums)
```

replace_it left **nums** alone; **empty_it** emptied it



Output

```
[1, 2]
[]
```

TRY IT: LISTS AND FUNCTIONS

A · Predict, then fix

```
1 def add_item(bag, item):
2     bag.append(item)
3     return bag
4
5 mine = ['pen']
6 yours = add_item(mine, 'cup')
7 print(mine)
8 print(yours is mine)
```

Fix **add_item** so **mine** stays **['pen']**.

B · Predict the output

```
1 def grow(a, b):
2     a += [9]
3     b = b + [9]
4
5 x = [1]
6 y = [1]
7 grow(x, y)
8 print(x, y)
```

Which parameter reached the caller's list? Why?

SUMMARY

Code	New outer list?	New inner lists?
<code>b = a</code>	No	No
<code>b = a.copy()</code> or <code>b = a[:]</code>	Yes	No, shared
<code>b = copy.deepcopy(a)</code>	Yes	Yes

`==` compares contents; `is` checks for the same object.

* on a list of lists repeats one inner list. Build rows with a comprehension.

In-place methods return **None**: never assign their result back to the list.